Grundlagen der Programmierung (Python) – kurz

Die Kurzfassung: nur das Wichtigste zu jedem Thema · Klausur **Fr 11.12.2026, 120 Minuten**

Auszug aus der langen Zusammenfassung (46 Seiten). Wenn dir ein Thema hier zu knapp ist, schlag es dort nach. Jede Ausgabe in den blauen Kästen kommt aus einem echten Lauf mit Python 3.13.

So lernst du damit

1. **Ein Block am Stück.** Lies zuerst den Satz mit dem Bild. Dann deckst du die blaue Ausgabe ab und sagst sie vorher. Erst danach schaust du hin.
2. **Schleifen immer mit Rundentabelle.** Eine Zeile pro Runde, eine Spalte pro Variable. Klingt langsam, aber so passieren keine Fehler, auch nicht in der Klausur.
3. **Die roten Kästen sind die Punkte-Fallen.** In „Was gibt das aus?“-Aufgaben steckt fast immer genau so eine Falle.
4. **Selbst tippen.** Schreib jedes Beispiel einmal von Hand ab (ohne IDE, wie im Kurs) und lass es mit `uv run datei.py` laufen. Ändere dann eine Zahl und rate die neue Ausgabe.
5. **Fehlermeldungen lesen.** Die letzte Zeile sagt, was falsch ist, die Zeile darüber, wo. Die Tabelle auf Seite 8 übersetzt sie.
6. **Am Ende nur noch der Spickzettel.** Wenn du jede Zeile darauf erklären kannst, bist du gut vorbereitet.

Mini-Quiz zum Start: Was gibt das aus? (Lösungen auf der letzten Seite)

1

```
print(7 // 2, 7 % 2)
```

2

```
s = 0
for i in range(1, 4):
    s = s + i
print(s)
```

3

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

4

```
def f(x):
    print(x * 2)

y = f(3)
print(y)
```

5

```
x = 15
if x > 5:
    print("A")
elif x > 10:
    print("B")
else:
    print("C")
```

Mach das Quiz jetzt einmal und am Ende noch einmal. Siehst du den Unterschied, hast du etwas gelernt.

1 Variablen und Datentypen

Bild: Eine Variable ist ein **Namensschild**, das du an einen Wert klebst. Der Typ sagt, was für ein Wert es ist: ganze Zahl (`int`), Kommazahl (`float`), Text (`str`) oder ja/nein (`bool`).

```
alter = 19              # int
preis = 2.5             # float
name = "Kaffee"         # str
offen = True            # bool
print(type(alter), type(preis))
print(7 / 2, 7 // 2, 7 % 2)
print(int("42") + 1, str(42) + "1")
print(f"{name} kostet {preis:.2f} Euro")
```

Ausgabe

```
<class 'int'> <class 'float'>
3.5 3 1
43 421
Kaffee kostet 2.50 Euro
```

Operator	heißt	Beispiel
`/`	teilen, immer float	`7 / 2` → 3.5
`//`	ganzzahlig teilen (abrunden)	`7 // 2` → 3
`%`	Rest beim Teilen	`7 % 2` → 1

Bild für `//` und `%`: 7 Kekse auf 2 Kinder. Jedes bekommt 3 (`//`), 1 bleibt übrig (`%`). `x % 2 == 0` heißt „x ist gerade“.

Fallen

- `"Alter: " + 19` → `TypeError`. Text und Zahl erst angleichen: `str(19)` oder f-String.
- `input()` liefert immer `str`: `int(input())`, sonst rechnest du mit Text (`"4" + "2"` → `"42"`).

2 Verzweigung: `if` / `elif` / `else`

Bild: Eine **Weiche** am Gleis: Python prüft die Bedingungen von oben nach unten und fährt in den **ersten** Zweig, der wahr ist. Alle anderen werden übersprungen.

```
punkte = 72
if punkte >= 90:
    print("sehr gut")
elif punkte >= 50:
    print("bestanden")
else:
    print("nicht bestanden")
```

Ausgabe

```
bestanden
```

Fallen

- **Reihenfolge:** Stünde `punkte >= 50` oben, käme bei 95 Punkten auch „bestanden“ heraus. Die strengste Bedingung gehört nach oben.
- Doppelpunkt am Zeilenende vergessen → `SyntaxError`. Block nicht eingerückt → `IndentationError`.
- `=` weist zu, `==` vergleicht.

3 `for`-Schleife mit `range` und mit Liste

Bild: Die **Kassiererin**: Sie nimmt eine Ware vom Band (eine Runde), scannt sie und nimmt die nächste. Ist das Band leer, sagt sie die Summe an. Die Schleifenvariable ist ihre Hand, in jeder Runde liegt etwas anderes darin.

```
summe = 0
for zahl in range(1, 4):
    summe = summe + zahl
    print("Runde:", zahl, "summe:", summe)
print("fertig:", summe)
```

Ausgabe

```
Runde: 1 summe: 1
Runde: 2 summe: 3
Runde: 3 summe: 6
fertig: 6
```

Rundentabelle zum Beispiel links oben (`range(1, 4)` liefert 1, 2, 3):

Runde	`zahl`	`summe` vorher	`summe` nachher
1	1	0	0 + 1 = 1
2	2	1	1 + 2 = 3
3	3	3	3 + 3 = 6
nach der Schleife (nicht eingerückt): einmal `fertig: 6`			

```
einkauf = ["Kaffee", "Tee", "Kakao"]
for ware in einkauf:
    print("Ich kaufe", ware)
```

Ausgabe

```
Ich kaufe Kaffee
Ich kaufe Tee
Ich kaufe Kakao
```

Fallen

- `range(1, 4)` → 1, 2, 3. **Die Stoppzahl ist nie dabei.** „1 bis 5“ heißt `range(1, 6)`.
- `summe = 0` **in** der Schleife setzt jede Runde zurück. Startwert immer **vor** die Schleife.
- Eingerückt = läuft jede Runde. Nicht eingerückt = läuft einmal danach.

4 `while`-Schleife

Bild: Ein **Kaffeeautomat**: Solange dein Guthaben reicht, kaufst du noch einen. Du weißt vorher nicht, wie viele Runden es werden. Vor jeder Runde wird die Bedingung neu geprüft.

```
guthaben = 10
while guthaben >= 4:
    guthaben = guthaben - 4
    print("Kaffee gekauft, Rest:", guthaben)
print("Zu wenig Geld")
```

Ausgabe

```
Kaffee gekauft, Rest: 6
Kaffee gekauft, Rest: 2
Zu wenig Geld
```

Prüfung	`guthaben >= 4`?	danach
1	10 → ja	6
2	6 → ja	2
3	2 → nein	Schleife endet

Fallen

- Die Bedingung ändert sich nie (z. B. `guthaben - 4` ohne Zuweisung) → **Endlosschleife**.
- `for`, wenn du weißt, wie oft (Liste, `range`). `while`, wenn es „bis etwas passiert“ heißt. `break` verlässt die Schleife sofort.

5 Listen

Bild: Eine Liste ist eine **Reihe nummerierter Schließfächer**. Die Nummerierung beginnt bei **0**, das letzte Fach erreichst du mit `-1`.

```
l = ["a", "b", "c"]
l.append("d")
print(l[0], l[-1], len(l))
print(l[1:3])
gitter = [[0] * 3] * 2      # FALLE
gitter[0][0] = 9
print(gitter)
gut = [[0] * 3 for _ in range(2)]
gut[0][0] = 9
print(gut)
```

Ausgabe

```
a d 4
['b', 'c']
[[9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0]]
```

- `l[0]` erstes, `l[-1]` letztes Element
- `l.append(x)` hängt hinten an, `len(l)` zählt
- Slicing `l[1:3]`: von Platz 1 bis **vor** Platz 3 (wie bei `range`)

Fallen

- `l[len(l)]` → `IndexError`. Der größte Index ist `len(l) - 1`.
- **Alias-Falle:** `[[0] * 3] * 2` enthält zweimal **dieselbe** innere Liste. Ändern einer Zeile ändert beide. Genauso bei `b = a`: zwei Namensschilder, eine Liste. Kopie: `a.copy()`.

6 Funktionen

Bild: Eine Funktion ist eine **Kaffeemaschine**: Du gibst etwas hinein (Parameter), sie arbeitet, und mit `return` kommt ein Ergebnis in deine Tasse. `print` dagegen zeigt nur etwas auf dem Display, in der Tasse landet nichts (`None`).

```
def brutto(netto: float, satz: float = 0.19) ->
float:
    return netto * (1 + satz)

def zeige(x: int) -> None:
    print(x * 2)

print(brutto(100))
print(brutto(100, 0.07))
ergebnis = zeige(5)
print(ergebnis)
```

Ausgabe

```
119.0
107.0
10
None
```

```
def rechne() -> None:
    zwischen = 5     # lokal: lebt nur in der
Funktion
    print(zwischen)

rechne()
print(zwischen)
```

Ausgabe

```
5
NameError: name 'zwischen' is not defined
```

- **Parameter** = Platzhalter in `def`, **Argument** = konkreter Wert beim Aufruf.
- **Standardwert** `satz=0.19`: Wenn du nichts angibst, gilt 0.19. Parameter mit Standardwert stehen **hinten**.
- **Gültigkeit:** Variablen aus der Funktion sind lokal. Sie verschwinden nach dem Aufruf (unten: `NameError`).
- `return` beendet die **ganze** Funktion sofort, auch mitten in einer Schleife.

Fallen

- **`print` ≠ `return`:** `zeige(5)` druckt 10, liefert aber `None`. Wer damit weiterrechnet, bekommt `TypeError … 'NoneType'`.
- `def` allein tut nichts. Der Rumpf läuft erst beim Aufruf `name(...)`.

7 Rekursion

Bild: **Matroschka-Puppen**: Jede Puppe öffnet die nächstkleinere, bis die kleinste (der **Basisfall**) nicht mehr aufgeht. Dann wird von innen nach außen wieder zugemacht. Eine rekursive Funktion ruft sich selbst mit einem kleineren Problem auf.

```
def fakultaet(n: int) -> int:
    if n <= 1:                    # Basisfall
        return 1
    return n * fakultaet(n - 1)   # kleineres
Problem

print(fakultaet(3))
print(fakultaet(5))
```

Ausgabe

```
6
120
```

Fallen

- Basisfall fehlt oder wird nie erreicht (z. B. `fakultaet(n)` statt `n - 1`) → `RecursionError`.
- Basisfall **zuerst** prüfen, erst dann rekursiv aufrufen.

Aufruf-Treppe für `fakultaet(3)`:

```
fakultaet(3) = 3 * fakultaet(2)        ↓ wartet
   fakultaet(2) = 2 * fakultaet(1)     ↓ wartet
      fakultaet(1) = 1                 Basisfall!
   fakultaet(2) = 2 * 1 = 2            ↑ rechnet
fakultaet(3) = 3 * 2 = 6               ↑ Ergebnis 6
```

Hinunter: jeder Aufruf wartet auf den nächsten (jeder hat sein eigenes `n`). Hinauf: erst jetzt wird multipliziert.

8 Klassen und Objekte

Bild: Eine Klasse ist die **Ausstechform**, die Objekte sind die **Plätzchen**. Jedes Plätzchen hat eigene Werte (Attribute) und kann etwas tun (Methoden). `self` heißt „dieses eine Plätzchen hier“.

```
class Konto:
    anzahl = 0                        # Klassenattribut

    def __init__(self, inhaber: str, stand: int = 0) -> None:
        self.inhaber = inhaber        # Objektattribut
        self._stand = stand
        Konto.anzahl += 1

    @property
    def stand(self) -> int:           # lesen ohne ()
        return self._stand

    def einzahlen(self, betrag: int) -> None:
        self._stand += betrag

a = Konto("Anna")
b = Konto("Ben", 50)
a.einzahlen(30)
print(a.inhaber, a.stand, b.stand, Konto.anzahl)
a.stand = 1000
```

Ausgabe

```
Anna 30 50 2
AttributeError: property 'stand' of 'Konto' object has no
setter
```

- `__init__` läuft automatisch bei `Konto("Anna")` und füllt das neue Objekt.
- `self.inhaber` gehört dem Objekt, `anzahl` (ohne `self`, direkt in der Klasse) teilen sich alle. Ändern über `Konto.anzahl`.
- **Property**: `a.stand` sieht aus wie ein Attribut, ruft aber eine Methode auf. Ohne Setter ist es nur lesbar (letzte Zeile: `AttributeError`).

```
from dataclasses import dataclass

@dataclass
class Kunde:
    name: str
    email: str

k = Kunde("Anna", "a@x.de")
print(k)
print(k == Kunde("Anna", "a@x.de"))
```

Ausgabe

```
Kunde(name='Anna', email='a@x.de')
True
```

`@dataclass` schreibt `__init__`, eine lesbare Ausgabe und `==` für dich.

Fallen

- `self` als ersten Parameter vergessen → `TypeError` beim Aufruf.
- `__str__` muss `return`en, nicht `print`en.

9 Vererbung

Bild: Ein Sparkonto **ist ein** Konto, nur mit Zins dazu. Die Unterklasse erbt alles von der Oberklasse und darf Einzelnes **überschreiben**, so wie ein Kind die Rezepte der Eltern übernimmt und eines davon nachwürzt.

```
from typing import override

class Konto:
    def __init__(self, inhaber: str) -> None:
        self.inhaber = inhaber

    def __str__(self) -> str:
        return f"Konto von {self.inhaber}"

class Sparkonto(Konto):                  # Sparkonto IST EIN
Konto
    def __init__(self, inhaber: str, zins: int) -> None:
        super().__init__(inhaber)        # Oberklasse zuerst!
        self.zins = zins

    @override
    def __str__(self) -> str:            # ueberschreiben +
ergaenzen
        return f"{super().__str__()} ({self.zins} %)"

s = Sparkonto("Anna", 2)
print(s)
print(isinstance(s, Konto), isinstance(Konto("Ben"),
Sparkonto))
```

Ausgabe

```
Konto von Anna (2 %)
True False
```

- `class Sparkonto(Konto)`: in Klammern steht die Oberklasse.
- `super().__init__(inhaber)` lässt zuerst die Oberklasse ihre Attribute anlegen.
- **Überschreiben:** gleiche Methode, gleicher Name, in der Unterklasse. `super().__str__()` holt die alte Version dazu. `@override` (aus `typing`) markiert das, `ty` meldet dann Tippfehler im Namen.
- `isinstance(s, Konto)` → `True`: ein Sparkonto zählt auch als Konto. Umgekehrt nicht.

Fallen

- `super().__init__(...)` vergessen → `AttributeError: … has no attribute 'inhaber'`.
- `super.__init__` ohne Klammern hinter `super` → `TypeError`.

10 Polymorphie und abstrakte Klassen

Bild: An der Kasse sagst du immer nur „Bezahlen bitte“. Ob dann Karte oder Rechnung läuft, entscheidet das Objekt, nicht du. Das ist **Polymorphie**. Die abstrakte Klasse ist das **Formular mit Pflichtfeld**: Jede Zahlungsart muss `bezahlen` ausfüllen, das leere Formular selbst kann man nicht benutzen.

```
from abc import ABC, abstractmethod

class Zahlungsart(ABC):
    @abstractmethod
    def bezahlen(self, betrag: float) -> str: ...

class Kreditkarte(Zahlungsart):
    def bezahlen(self, betrag: float) -> str:
        return f"{betrag} Euro per Karte"

class Rechnung(Zahlungsart):
    def bezahlen(self, betrag: float) -> str:
        return f"{betrag} Euro auf Rechnung"

for art in [Kreditkarte(), Rechnung()]:   # gleicher Aufruf,
    print(art.bezahlen(9.5))               # anderes Verhalten
Zahlungsart()
```

Ausgabe

```
9.5 Euro per Karte
9.5 Euro auf Rechnung
TypeError: Can't instantiate abstract class Zahlungsart without
an implementation for abstract method 'bezahlen'
```

- Abstrakt braucht **zwei Zutaten**: erben von `ABC` und `@abstractmethod` über der Pflichtmethode. Import: `from abc import ABC, abstractmethod`.
- Die Schleife ruft überall dasselbe `art.bezahlen(...)` auf. Welche Version läuft, hängt vom Objekt ab.
- Neue Zahlungsart = neue Unterklasse. Der Code mit der Schleife bleibt gleich.

Fallen

- Abstrakte Klasse erzeugen oder Pflichtmethode in der Unterklasse vergessen → `TypeError: Can't instantiate abstract class …`
- `@abstractmethod` ohne `(ABC)` → kein Fehler, nichts wird geprüft.

11 Projekt Kaffeeshop in 5 Zeilen

Bild: Das Lager ist ein Regal voller Produkte. Im Warenkorb liegen keine Produkte, sondern **Zettel** („3 × French Press“). Das sind die Positionen.

Baustein	in einem Satz
Lager	hat eine Liste von Produkten (*hat ein*, keine Vererbung).
Warenkorb	hat eine Liste von Positionen, am Anfang leer: `self.positionen = []`.
Position	ein Zettel aus Produkt und Menge (`@dataclass`).
`finden(nummer)`	Schleife über die Produkte. Treffer → sofort `return produkt`. Erst **nach** der Schleife `return None`.
`hinzufuegen(p, menge)`	Menge im Korb + neue Menge > Bestand → `False`. Liegt das Produkt schon im Korb, wird die Menge auf dem Zettel erhöht, sonst kommt ein neuer Zettel dazu → `True`.

```
from dataclasses import dataclass

@dataclass
class Produkt:
    nummer: int
    name: str
    bestand: int

@dataclass
class Position:                      # ein Zettel: Produkt + Menge
    produkt: Produkt
    menge: int

class Lager:
    def __init__(self, produkte: list[Produkt]) -> None:
        self.produkte = produkte

    def finden(self, nummer: int) -> Produkt | None:
        for produkt in self.produkte:
            if produkt.nummer == nummer:
                return produkt       # gefunden: sofort raus
        return None                  # erst NACH der Schleife

class Warenkorb:
    def __init__(self) -> None:
        self.positionen: list[Position] = []

    def hinzufuegen(self, produkt: Produkt, menge: int) -> bool:
        for pos in self.positionen:
            if pos.produkt.nummer == produkt.nummer:
                if pos.menge + menge > produkt.bestand:
                    return False
                pos.menge += menge   # vorhandenen Zettel erhoehen
                return True
        if menge > produkt.bestand:
            return False
        self.positionen.append(Position(produkt, menge))
        return True

lager = Lager([Produkt(101, "Kaffee", 40), Produkt(203, "French
Press", 3)])
korb = Warenkorb()
fp = lager.finden(203)
print(fp)
print(lager.finden(999))
if fp is not None:
    print(korb.hinzufuegen(fp, 3), korb.hinzufuegen(fp, 1))
print(len(korb.positionen), korb.positionen[0].menge)
```

Ausgabe

```
Produkt(nummer=203, name='French Press', bestand=3)
None
True False
1 3
```

Was passiert unten? `finden(203)` trifft in Runde 2. `finden(999)` trifft nie, also `None`. 3 French Press passen genau (3 ≤ 3), die vierte nicht mehr (3 + 1 > 3). Ergebnis: **eine** Position mit Menge 3.

Fallen

- `else: return None` direkt unter dem `if` in `finden()` → die Suche endet nach Runde 1, 203 wird nie gefunden.
- Ergebnis von `finden()` nicht auf `None` prüfen → `AttributeError: 'NoneType' object has no attribute …`
- Bei der Bestandsprüfung die Menge im Korb vergessen → 4 von 3 im Korb.
- Immer `append`, statt die Menge zu erhöhen → dasselbe Produkt steht zweimal im Korb.
- Methoden liefern `True` / `False` mit `return`. Die Meldungen für den Nutzer druckt `main.py`.

12 Typische Fehlermeldungen

Bild: Ein Traceback ist ein **Unfallbericht**. Lies ihn von unten: Die letzte Zeile sagt, **was** passiert ist, die Zeile darüber, **wo**.

Fehler	Ursache (typisch)
`SyntaxError`	`:` oder Klammer fehlt, `x++`, `=` statt `==` im `if`
`IndentationError`	Block nach `:` nicht (oder falsch) eingerückt
`NameError`	Name nie zugewiesen, Tippfehler, Groß/klein, lokale Variable außerhalb der Funktion, Import vergessen
`TypeError`	`"Alter: " + 19`, Argument fehlt, `self` vergessen, abstrakte Klasse erzeugt, mit `None` gerechnet
`ValueError`	`int("zwei")`: richtiger Typ, aber unpassender Inhalt
`IndexError`	Index zu groß, z. B. `l[len(l)]`
`ZeroDivisionError`	durch 0 geteilt
`AttributeError`	Attribut gibt es nicht: Tippfehler, `super().__init__` vergessen, `None.name`, Property ohne Setter beschrieben
`RecursionError`	Basisfall fehlt oder wird nie erreicht
kein Fehler, falsches Ergebnis	Logikfehler: Startwert in der Schleife, `range` eine Runde zu kurz, `print` statt `return`, falsche Reihenfolge bei `elif`

Spickzettel auf einer Seite

Typen und Operatoren

```
int 7 · float 3.5 · str "a" · bool True · None
7 / 2 → 3.5     7 // 2 → 3     7 % 2 → 1
2 ** 3 → 8      "ab" * 2 → "abab"
int("42") → 42  str(3) → "3"   float("2.5") → 2.5
x += 1   # x++ gibt es nicht
=  zuweisen    ==  vergleichen
f"{name} kostet {preis:.2f} Euro"
```

Verzweigung

```
if x > 10:        # strengste Bedingung oben
    ...
elif x > 5:       # erster wahrer Zweig gewinnt
    ...
else:
    ...
```

Schleifen

```
range(3) → 0 1 2      range(1, 4) → 1 2 3
range(0, 10, 3) → 0 3 6 9   (Stopp nie dabei)
summe = 0             # Startwert VOR die Schleife
for ware in einkauf:  # jedes Element einmal
    summe += ware
while guthaben >= 4:  # Bedingung muss sich ändern
    guthaben -= 4
break     # Schleife verlassen
continue  # nächste Runde
return    # ganze Funktion verlassen
```

Schleifenmuster

finden	Treffer → `return x`, nach der Schleife `return None`
sammeln	`treffer = []` → `append` → `return treffer`
summieren	`summe = 0` → `summe += …` → `return summe`
zählen	`anzahl = 0` → `anzahl += 1` → `return anzahl`

Listen

```
l = ["a", "b", "c"]
l[0] → "a"   l[-1] → "c"   l[1:3] → ["b", "c"]
l.append("d")   len(l) → 4   "a" in l → True
b = l          # derselbe Inhalt, 2 Namen!
c = l.copy()   # echte Kopie
[[0] * 3] * 2  # FALLE: 2 x dieselbe Zeile
[[0] * 3 for _ in range(2)]   # richtig
```

Funktionen und Rekursion

```
def brutto(netto: float, satz: float = 0.19) -> float:
    return netto * (1 + satz)
brutto(100) → 119.0   brutto(100, 0.07) → 107.0
# ohne return → None   (print ≠ return)
def fakultaet(n: int) -> int:
    if n <= 1:                  # Basisfall zuerst
        return 1
    return n * fakultaet(n - 1)
```

Klassen

```
class Konto:
    anzahl = 0                    # Klassenattribut
    def __init__(self, inhaber: str) -> None:
        self.inhaber = inhaber    # Objektattribut
        self._stand = 0
        Konto.anzahl += 1
    @property
    def stand(self) -> int:       # k.stand (ohne Klammern)
        return self._stand
    def __str__(self) -> str:     # return, nicht print
        return f"Konto von {self.inhaber}"

@dataclass                        # __init__, Ausgabe, ==
gratis
class Kunde:
    name: str
    email: str
```

Vererbung, Abstraktion

```
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import override

class Sparkonto(Konto):           # ist ein Konto
    def __init__(self, inhaber: str, zins: int) -> None:
        super().__init__(inhaber) # Pflicht!
        self.zins = zins
    @override
    def __str__(self) -> str:
        return super().__str__() + " mit Zins"

class Zahlungsart(ABC):           # ABC + abstractmethod
    @abstractmethod
    def bezahlen(self, betrag: float) -> str: ...

isinstance(Sparkonto("A", 2), Konto) → True
```

Fehler in einem Satz

`SyntaxError`	`:` oder Klammer fehlt
`IndentationError`	Block nicht eingerückt
`NameError`	Name unbekannt, Import fehlt
`TypeError`	falscher Typ, Argument fehlt, abstrakt erzeugt, `None + 1`
`ValueError`	`int("zwei")`
`IndexError`	Index zu groß
`AttributeError`	Attribut fehlt, `super().__init__` vergessen, `None.name`
`RecursionError`	Basisfall nie erreicht

Lösungen zum Mini-Quiz von Seite 1

Nr.	Ausgabe	Warum
1	**Ausgabe** `3 1`	7 Kekse auf 2 Kinder: jedes bekommt 3 (`//`), 1 bleibt übrig (`%`).
2	**Ausgabe** `6`	`range(1, 4)` ist 1, 2, 3 (ohne 4). Rundentabelle: `s` = 1, 3, 6.
3	**Ausgabe** `[1, 2, 3]`	`b = a` kopiert nicht. Zwei Namensschilder, eine Liste.

4	**Ausgabe**
	`6`
	`None`
		`f` druckt 6, hat aber kein `return`, also ist `y` gleich `None`.

5	**Ausgabe** `A`	15 > 5 ist schon wahr, der erste wahre Zweig gewinnt. `B` kommt nie dran, obwohl 15 > 10 auch stimmt.